



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section No.: 11

Team No.: 1

Project Title: Linux Concurrent Producer-Consumer System Using Semaphores

Team Members

Roll Number	Student Name	Signature
2520090169	Bhargava	
2520090107	Hanish	

1. Abstract

This project implements a Linux Concurrent Producer-Consumer System using POSIX semaphores and POSIX threads (pthreads) as the primary synchronization and concurrency mechanism. The producer-consumer problem is a classical operating systems challenge that arises when multiple concurrent processes or threads must share a bounded buffer: producers generate data items and place them into the buffer, while consumers retrieve and process those items. Without proper synchronization, race conditions, buffer overflows, and deadlocks can occur, leading to incorrect program behavior. The proposed solution implements a fixed-size shared circular buffer managed by multiple producer threads and multiple consumer threads running concurrently on Ubuntu Linux. Three POSIX semaphores are employed: empty to track available slots, full to track filled slots, and mutex as a binary semaphore to provide mutual exclusion to protect critical sections during buffer access. The system demonstrates core OS concepts including process synchronization, mutual exclusion, critical section management, and concurrent thread execution. The implementation is written in C using Linux/POSIX APIs such as `sem_init`, `sem_wait`, `sem_post`, `pthread_create`, and `pthread_join`. The project validates correctness through test cases that exercise boundary conditions, concurrent access patterns, and stress scenarios with varying producer and consumer counts. Expected outcomes include a functional, race-condition-free concurrent system with measurable throughput and a demonstration of how semaphore-based synchronization enforces orderly access to shared resources in a multi-threaded Linux environment.

2. Problem Statement

In multi-threaded and multi-process environments, concurrent access to shared resources is a fundamental challenge in operating systems design. The producer-consumer problem specifically addresses scenarios where one or more producer threads generate data and place it into a shared bounded buffer, while one or more consumer threads retrieve and process that data. The core problem is ensuring that producers do not write to a full buffer, consumers do not read from an empty buffer, and that no two threads access the shared buffer simultaneously in an unsafe manner. Without proper synchronization primitives, these violations lead to race conditions, data corruption, and non-deterministic program behavior. This project addresses the problem by implementing a correctly synchronized producer-consumer system on Linux using POSIX semaphores and mutexes, directly demonstrating the practical application of OS synchronization theory in systems programming.

3. Objectives

1. Implement a fixed-size shared circular buffer accessed concurrently by multiple producer and consumer POSIX threads.
2. Apply POSIX semaphores (`sem_init`, `sem_wait`, `sem_post`) and a mutex to enforce mutual exclusion and bounded-buffer constraints.
3. Demonstrate the prevention of race conditions, buffer overflow, and buffer underflow through correct synchronization.
4. Test and analyze system behavior under varying thread counts, buffer sizes, and workloads, and document experimental results.

4. Proposed Methodology

The project will be designed and implemented in C on Ubuntu Linux. A fixed-size circular buffer of configurable capacity will serve as the shared data structure. Three POSIX semaphores will be initialized: "empty" (initialized to buffer size) to count available slots, "full" (initialized to 0) to count occupied slots, and "mutex" (binary semaphore acting as a lock) to protect the critical section. Producer threads will call `sem_wait` on "empty" to block when the buffer is full, then acquire the mutex, insert an item, release the mutex, and call `sem_post` on "full". Consumer threads will call `sem_wait` on "full" to block when the buffer is empty, acquire the mutex, remove an item, release the mutex, and call `sem_post` on "empty". Multiple producer and consumer threads will be created using `pthread_create` and joined using `pthread_join`. The system will be tested with different configurations for correctness verification and throughput measurement. GDB and strace will be used for debugging and system call analysis.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
POSIX Semaphores (<code>sem_init</code> , <code>sem_wait</code> , <code>sem_post</code> , <code>sem_destroy</code>)	Synchronize producer and consumer threads; enforce buffer capacity constraints
POSIX Threads (<code>pthread_create</code> , <code>pthread_join</code> , <code>pthread_exit</code>)	Create and manage concurrent producer and consumer threads
Mutex / Binary Semaphore (<code>sem_t</code> as mutex)	Protect the critical section during shared buffer read/write operations
Bounded Buffer / Circular Buffer (shared memory array)	Shared data structure for inter-thread item exchange
Process Synchronization and Mutual Exclusion	Core OS concept demonstrated through semaphore-based locking
GDB / strace	Debugging thread behavior and tracing system calls during execution

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090169	Bhargava	System design, circular buffer implementation, producer thread logic, testing and results documentation
2520090107	Hanish	Semaphore initialization and synchronization logic, consumer thread logic, debugging, project report and demonstration

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Primary implementation and demonstration platform
C	Programming language for system-level implementation using POSIX APIs
POSIX Threads Library (pthreads)	Thread creation, management, and synchronization
POSIX Semaphore Library (semaphore.h)	Semaphore-based synchronization of producer and consumer threads
GDB (GNU Debugger)	Debugging multi-threaded execution and identifying race conditions
strace	Tracing Linux system calls during program execution for analysis
gcc / Makefile	Compilation and build automation of the C source code
GitHub	Version control and source code repository management

8. Expected Outcome

The project will deliver a fully functional, race-condition-free concurrent producer-consumer system on Linux. The system will correctly handle all boundary conditions: producers will block when the buffer is full and resume only when space is available; consumers will block when the buffer is empty and resume only when items are present. The implementation is designed to prevent data corruption, deadlock, and synchronization errors under the tested configurations. Experimental results will demonstrate measurable throughput under varying thread counts and buffer sizes. The final deliverable will include well-documented C source code, a Makefile, a README, test case outputs, and a project report mapping the implementation to core OS synchronization concepts.

9. GitHub Repository

GitHub Repository Link: <https://github.com/HanishDev0017/OS-Project>

Repository Created: Yes

Faculty Name and Signature

Faculty Name: Mrs. Alekhya

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____